

****RELATÓRIO TÉCNICO: ANÁLISE PRÁTICA DE PROCESSOS E THREADS EM PYTHON****

****1. INTRODUÇÃO****

O presente relatório técnico tem como objetivo analisar um código desenvolvido em linguagem Python, focado na biblioteca `threading`, e correlacionar suas instruções com os conceitos fundamentais apresentados na Unidade 2, Seção 1 do livro "Sistemas Operacionais", de autoria de Cynthia da Silva Barbosa. A seção aborda especificamente a definição, os estados, a hierarquia e o ciclo de vida de processos e threads gerenciados pelo sistema operacional.

****2. FUNDAMENTAÇÃO TEÓRICA****

Na teoria dos sistemas operacionais, um processo é fundamentalmente definido como um programa em execução. Para otimizar o uso da CPU, o processador alterna rapidamente a execução de vários programas, criando para o usuário a ilusão de que todas as tarefas estão sendo executadas simultaneamente, um fenômeno conhecido como pseudoparalelismo.

Para reduzir o tempo gasto na criação e na troca de contexto de processos, introduziu-se o conceito de **threads** (ou processos leves), que são fluxos de controle ou execução dentro de um mesmo processo que compartilham os seus recursos. Durante a sua execução, processos e **threads** passam por três estados fundamentais: ****Em execução**** (processados pela CPU), ****Pronto**** (aguardando a liberação da CPU) e ****Espera/Bloqueado**** (aguardando um evento externo para continuar).

****3. ANÁLISE DO CÓDIGO E CORRELAÇÕES TEÓRICO-PRÁTICAS****

O script em Python atua simulando um software de "Editor de Texto", utilizando a criação de duas **threads** para demonstrar o funcionamento interno do programa. A seguir, detalha-se a correlação de cada bloco com a teoria:

*** **Definição de Threads e o Exemplo do Processador de Texto:****

O código cria duas funções distintas, ``tarefa_digitacao`` e ``tarefa_auto_save``, que são alocadas em duas **threads** (``thread_1`` e ``thread_2``) que compartilham o mesmo processo pai. Essa implementação reflete exatamente o exemplo teórico da literatura, que ilustra um processador de texto utilizando **threads** simultâneas: uma encarregada de atender aos requisitos do usuário via teclado e outra responsável por fazer o backup dos dados em disco sem interferir na digitação.

*** **Pseudoparalelismo e Estados do Processo:****

No escopo da ``tarefa_digitacao``, a função ``input()`` é invocada. Teoricamente, isso força a **thread** a entrar no estado ****Espera ou Bloqueado****, pois o fluxo de execução é paralisado aguardando por um evento externo (o comando do usuário pelo teclado) para poder continuar.

Simultaneamente, a ``tarefa_auto_save`` utiliza o comando ``time.sleep()``. Esse comando no código simula o conceito de ****pseudoparalelismo****, representando o momento em que o escalonador do sistema operacional pausa aquela **thread** e passa a vez de execução para outro programa (ou **thread**) ativo na máquina.

*** **Ciclo de Vida e Sincronização (Tabela de Processos):****

O uso do método `.start()` instrui o sistema operacional a inserir as *threads* recém-criadas na fila de execução, iniciando ativamente seus ciclos de vida. Em seguida, os comandos `thread_1.join()` e `thread_2.join()` atuam na sincronização e no controle do ciclo de término. Eles garantem que o processo pai apenas seja finalizado (saída normal) após a conclusão total de suas *threads* filhas. Para gerenciar isso internamente, o sistema operacional mantém um "quadro de processos" (ou tabela de processos), salvando os estados de cada fluxo para que eles retomem do ponto exato em que pararam e sincronizem o encerramento do processo geral.

****4. CONCLUSÃO****

O código analisado ilustra na prática a utilidade de se programar de forma concorrente utilizando *threads*. A divisão do programa em fluxos independentes evidencia os conceitos de estados e de gerenciamento do ciclo de vida explicados pela teoria, demonstrando como uma aplicação real de edição de texto consegue receber a digitação do usuário e formatar/salvar o arquivo simultaneamente, economizando recursos e otimizando o desempenho através do pseudoparalelismo gerido pelo Sistema Operacional.

****5. REFERÊNCIAS BIBLIOGRÁFICAS****

BARBOSA, Cynthia da Silva. ****Sistemas Operacionais****. Londrina: Editora e Distribuidora Educacional S.A., 2018.